SISTEMA BANCÁRIO EM JAVA

Relatório técnico do projeto

Trabalho de Programação em Java

Instituição: unifesspa
Disciplina: estrutura de dados
Professor(a): Adam
Integrantes: Mateus Torquato, Diogo Lemos, Miguel Damasceno, Sandro Nascimento, Isaac Daniel

Novembro de 2026

Resumo

O Sistema Bancário em Java é uma aplicação de terminal para administrar clientes, contas e transações. O projeto combina programação orientada a objetos, validação de entradas, regras de movimentação financeira, fechamento mensal, busca e consulta de informações, persistência em arquivos de texto e geração de relatórios. Os dados são organizados de forma hierárquica: o banco mantém clientes; cada cliente possui contas; cada conta registra transações.

Na verificação realizada para este material, o projeto foi compilado com OpenJDK 21.0.12.1 e a classe TestesFinais concluiu nove grupos de testes, sem falhas. A documentação do projeto estabelece JDK 17 ou superior como requisito mínimo.

1. Introdução e objetivo

O trabalho propõe um sistema bancário simplificado executado no terminal. Seu objetivo é aplicar conceitos de Java em uma situação prática, organizando dados e operações por meio de classes, métodos, listas e enumerações. A aplicação permite cadastrar clientes, associar contas, registrar movimentações, consultar saldos e extratos, gerar relatórios e manter os dados salvos entre execuções.

O sistema é acadêmico e simula regras bancárias básicas. Ele não é uma aplicação bancária comercial nem realiza integração com instituições financeiras reais.

2. Organização do domínio

A estrutura central é hierárquica. A classe Banco mantém uma coleção de Cliente. Cada Cliente mantém uma coleção de Conta. Cada Conta mantém uma lista encadeada de Transacao. Essa organização expressa a relação entre os objetos e facilita percorrer os dados para consultas e relatórios.

Elemento	Principais dados / função
Banco	Nome e lista de clientes; busca por CPF e número de conta; transferências e fechamento mensal.
Cliente	CPF, nome e lista de contas; inclusão/remoção de conta e cálculo do saldo total.
Conta	Número, tipo, saldo inicial e transações; registro, saldo, limite, saldo por data e extrato.
Transacao	Data, tipo, valor e descrição; indica se o lançamento é crédito ou débito.
TipoConta	Enum com CORRENTE e POUPANCA.
TipoTransacao	Enum com DEPOSITO, SAQUE, PIX_ENVIADO, PIX_RECEBIDO, TARIFA e RENDIMENTO.

O fluxo pode ser lido assim: Banco → Cliente → Conta → Transacao. Os enums restringem os tipos a opções previstas no código e diminuem erros de digitação nos tipos de conta e de lançamento.

3. Responsabilidade das classes

Classe	Responsabilidade no projeto
Main	Apresenta o menu, recebe entradas, valida dados, pede confirmações e encaminha as ações às classes do domínio.
Banco	Gerencia clientes, impede duplicidades, localiza clientes/contas, processa transferências e fechamento mensal.
Cliente	Guarda identificação e contas; permite adicionar/remover conta sob condições e calcular saldo consolidado.
Conta	Registra transações em ordem cronológica, calcula saldo, aplica limite, calcula saldo em uma data e monta extrato.
Transacao	Representa o lançamento financeiro e classifica crédito ou débito.
LeitorDeArquivo	Lê e valida o arquivo de dados para reconstruir o objeto Banco.
GravadorDeArquivo	Serializa os dados do banco em formato textual.
GeradorRelatorio	Produz relatórios de clientes, contas, transações e extratos mensais.
TestesFinais	Executa verificações automatizadas sem depender de JUnit.

4. Interface de terminal e validação

A classe Main mantém o menu em um laço de repetição até que o usuário escolha sair. O menu reúne as operações de listagem, detalhamento, cadastro, inclusão e remoção de elementos, busca, operações bancárias, relatórios, salvamento e saída. Submenus agrupam tarefas relacionadas, como operações bancárias e relatórios.

Método de entrada	Validação realizada
lerInteiro()	Converte o texto para inteiro. Se a conversão falhar, captura NumberFormatException e repete a pergunta.
lerDouble()	Converte para double e rejeita valores não finitos ou menores/iguais a zero.
lerSaldoInicial()	Permite zero, mas rejeita valores negativos e não finitos.
lerData()	Usa LocalDate e DateTimeFormatter com resolução estrita no padrão dd/MM/uuuu.
lerCPF()	Exige exatamente 11 caracteres numéricos. Não verifica os dígitos verificadores oficiais.
lerTexto()	Rejeita texto vazio ou composto somente por espaços.
confirmar()	Aceita s ou n, sem diferenciar maiúsculas de minúsculas; repete em outras respostas.

A estratégia é separar erros de formato de erros de regra. Por exemplo, digitar letras quando se espera um inteiro é tratado por try-catch; digitar um número que não corresponde a uma opção válida é tratado pelo default do switch.

5. Operações e regras de negócio

Operação / regra	Comportamento implementado
Depósito	Registra um crédito na conta, com valor positivo e data informada.
Saque	Registra débito somente quando o saldo disponível mais o limite permite a operação.
PIX	Registra PIX_ENVIADO na origem e PIX_RECEBIDO no destino. Se o destino recusar o crédito, o débito de origem é desfeito.
Limite	Conta CORRENTE possui limite de R$ 500,00; POUPANCA não possui limite adicional.

Operação / regra	Comportamento implementado
Ordem cronológica	Uma conta não aceita transação com data anterior à sua última transação registrada.
Remoção de cliente	O cliente só pode ser removido se não possuir contas.
Remoção de conta	A conta precisa apresentar saldo zero; a classe Cliente executa a remoção.
Estorno	O método estornarUltima retira a última transação, mas não permite remover TARIFA ou RENDIMENTO.

6. Fechamento mensal

O método fecharMesComResumo(ano, mes), na classe Banco, aplica uma regra de acordo com o tipo de conta. Para CORRENTE, registra tarifa mensal de R$ 15,00. Para POUPANCA, calcula rendimento de 0,5% sobre o saldo positivo, com arredondamento para duas casas decimais.

- O fechamento verifica se o lançamento correspondente já existe no mês, evitando cobrança ou rendimento duplicado.
- Conta poupança sem saldo positivo não recebe rendimento.
- Uma conta corrente pode ter a tarifa recusada se saldo e limite forem insuficientes.
- Fechamentos retroativos são recusados quando há transação posterior ao último dia do mês, preservando a ordem cronológica.
- A operação devolve resumo com processadas, duplicadas, recusadas e contas sem rendimento aplicável.

7. Persistência e formato dos arquivos

Os dados são armazenados em texto. Ao iniciar, Main verifica a existência de dados/banco.txt e tenta carregá-lo com LeitorDeArquivo. Quando o usuário escolhe salvar, GravadorDeArquivo grava o estado atual nesse caminho. Assim, os dados não dependem apenas da memória do programa.

O formato é organizado em linhas: primeiro o nome do banco; em seguida, para cada cliente, CPF, nome e quantidade de contas; para cada conta, número, tipo, saldo inicial e quantidade de transações; depois, cada transação é registrada com data, tipo, valor e descrição. Os campos são separados por ponto e vírgula; valores numéricos usam ponto decimal.

O leitor verifica quantidade de campos, dados vazios, CPF, duplicidades, números, datas, enums e a validade das transações. O arquivo de dados grande é mantido separado do arquivo de persistência normal para que os testes não substituam os dados do usuário.

8. Relatórios

A classe GeradorRelatorio gera arquivos na pasta saidas/ e apresenta o conteúdo no terminal. O projeto disponibiliza relatório de clientes, relatório de contas, relatório de transações e extratos mensais por conta.

Saída	Conteúdo
saidas/clientes.txt	CPF, nome, quantidade de contas e saldo total por cliente.
saidas/contas.txt	Número da conta, titular, tipo e saldo.
saidas/transacoes.txt	Titular, conta, data, valor, tipo e descrição de cada lançamento.
saidas/extrato_<conta>_<mês>_<ano>.txt	Saldo anterior, transações do mês e saldo final.

9. Conjunto de dados em massa

O arquivo dados/grande.txt foi preparado para testar o carregamento de um volume maior de informações. Ele contém 500 clientes, 1.000 contas e 3.000 transações, distribuídas em 4.501 linhas incluindo o nome do banco. A base não é carregada automaticamente na inicialização normal; é usada pelo teste de leitura em massa ou pode ser carregada manualmente pelo leitor.

10. Testes e resultados

A classe TestesFinais contém nove grupos de testes automatizados. No ambiente verificado para este material, o projeto compilou e os nove grupos foram aprovados, sem falhas.

Grupo de teste	O que verifica
1. Validações de entrada	Inteiro, decimal, data, confirmação e CPF.
2. Cadastro e valores	CPF e conta duplicados; saldo e dados inválidos.
3. Movimentações	Depósito, saque, limite e ordem cronológica.
4. Transferência PIX	Sucesso, saldo de origem/destino e reversão em caso de falha.
5. Fechamento mensal	Tarifa, rendimento, duplicidade, recusas e mês inválido.
6. Persistência	Salvar e carregar sem perder contas, transações e saldos.
7. Base grande	Carregar 500 clientes, 1.000 contas e 3.000 transações.
8. Relatórios	Gerar relatórios e extrato mensal.
9. Arquivo inválido	Rejeitar estrutura de dados malformada.

Resultado verificado: 9 aprovados e 0 falhas. O resultado refere-se à execução da classe TestesFinais no ambiente OpenJDK 21.0.12.1 desta preparação.

11. Compilação e execução

Requisito indicado no README: JDK 17 ou superior. Os comandos abaixo devem ser executados na pasta raiz do projeto.

Windows (PowerShell ou CMD):

```
mkdir out
javac -encoding UTF-8 -d out src\*.java
java -cp out Main
```

Linux/macOS:

```
mkdir -p out
javac -encoding UTF-8 -d out src/*.java
java -cp out Main
```

Para executar os testes depois da compilação:

```
java -cp out TestesFinais
```

12. Limites do escopo

- A interface é textual, executada no terminal; o projeto não apresenta interface gráfica.
- O CPF é validado pelo formato de 11 dígitos, sem validação matemática dos dígitos verificadores.
- A persistência usa arquivos de texto locais, não um banco de dados relacional nem serviço remoto.
- O projeto é uma simulação acadêmica e não está conectado a sistemas bancários reais.

13. Conclusão

O projeto integra conceitos fundamentais de Java em uma aplicação coesa. A separação entre entidades, regras de negócio, interface, persistência, relatórios e testes facilita a leitura e a manutenção do código. Além das operações básicas, o trabalho demonstra tratamento de entradas inválidas, validação de regras bancárias, prevenção de duplicidades, processamento mensal e recuperação dos dados após reinicialização.

Referências do próprio projeto

- Código-fonte: Banco.java, Cliente.java, Conta.java, Transacao.java, TipoConta.java e TipoTransacao.java.
- Interface e integração: Main.java.
- Persistência: LeitorDeArquivo.java e GravadorDeArquivo.java.
- Relatórios: GeradorRelatorio.java.
- Validação: TestesFinais.java, README.md, dados/grande.txt e RESULTADO_TESTES.txt.

Link repositório github: https://github.com/Isaac-BrDev/TrabalhoEDcodigo